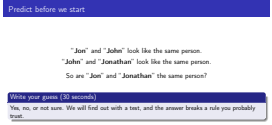
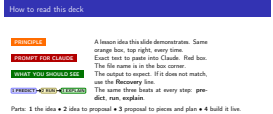


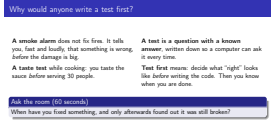

Say: Today I am going to show you the whole process, start to finish, the way people really build software: an idea, a written proposal, a plan, and then working code with tests. I will use Claude to write the Python live, and I will not pretend it goes perfectly. You will see red tests. That is the point. On Thursday you repeat this same process yourselves on a different problem. Nothing today is a trick; every step has a plain question for you before I run it.




Do this before anything else. Take a show of hands: yes, no, not sure. Most will say yes. Do not reveal. Say: by the end of Part 4 we will have a test that answers this with a concrete example, and we will see why the answer matters for a program that merges duplicate records. The answer, for your eyes: with our first matching rule Jon matches John and John matches Jonathan but Jon does NOT match Jonathan (step 5 prints False, step 6 catches it). That failure of the chain is what transitive means.




Walk the legend once, then stop talking about it. Every slide in Part 4 follows the same layout so the room learns the pattern in two steps and stops thinking about the slide. If you lose your place, find the orange box (what idea is this), the red box (what do I paste), the green line (what should I see).




Two minutes, no code. Smoke alarm: nobody thinks a smoke alarm is an insult to the house; it is cheap insurance. Taste test: you would never serve soup you have not tasted; a test is tasting the code. Then the key sentence: a test is a question with a known answer. Examples a student can relate to: checking your answer by plugging it back into the equation; proofreading. Expected answers to the question: submitted homework with a typo in the formula, code that worked on my laptop and not the teacher's, a recipe that burned. Do not define unit test yet; the glossary slide does that after they have the feeling.




This is the most important slide for this audience. Do the loop out loud for the first three steps even if it feels slow. Predict: give 30 seconds and take two guesses from the room before you press enter. Run: press enter, read the output aloud, line by line, once. Explain: ask one student for one sentence beginning with the word because. If the prediction was wrong, praise the wrong prediction: it showed us something. Common failure: skipping predict when pressed for time. Do not skip it; shorten the step instead.

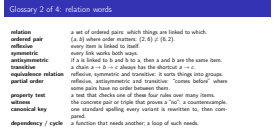



Name the feeling out loud: red can feel like I got it wrong. It does not. Red is the alarm working. Tell the story: a smoke alarm that has never gone off in a drill might be dead, and you would not know. Practical tip for reading pytest output: look at the bottom line first (X failed, Y passed), then find the line starting with E, which says why. Expected: students will still flinch at red on Thursday. Normalize it by saying it each time, calmly.

Glossary 1 of 4: testing words

| | |
|---|---|
| **test** | a question with a known answer, written so a computer can check it. |
| **unit test** | a test for one small piece of code, a single function. |
| **pytest** | the tool that runs our tests and shows the results. |
| **assert** | the line that says "this must be true"; if not, the test fails. |
| **red / green** | red: a test is failing. green: all tests pass. |
| **test-driven development (TDD)** | write a failing test first, then just enough code to make it pass. |
| **fixture** | fixed sample data a test uses, such as a list of names. |
| **stub** | an empty placeholder function so a test can run and fail. |
| **partition (bin)** | a group of inputs the code treats the same way. |
| **boundary** | the edge between two bins, such as 59 and 60. |

Do not read this slide. Leave it up while students ask. First time each word appears, say the one-line meaning and point back here. Handout has the same list. If someone asks what is the difference between a test and a unit test: a unit test is just a small, focused test of one function; that is the only kind we write today.




Same approach: leave it up, do not read it. The seating chart in Part 1 supplies the concrete examples for each of these before the formal word is used. If a student says that is a lot of words: yes, and we will use each one on a real example within ten minutes. Union-find and edit distance are defined where they appear in Part 4.

Glossary 3 of 4: words in the steps

| | |
|---|---|
| **counterexample** | one concrete case that shows a rule is false; same idea as a witness. |
| **matrix** | a grid of 0s and 1s: row linked to column, 1 means yes. |
| **digraph** | dots joined by arrows; one arrow per ordered pair. |
| **edit distance** | how many single-letter inserts, deletes or changes turn one word into another. |
| **normalize** | rewrite many spellings into one standard form before comparing. |
| **union-find** | a way to group items: each match merges two groups. Also called connected components. |
| **leaf** | a function that needs no other function; test it first. |
| **property** | a rule that must hold for every item, such as symmetric. |
| **PRINCIPLE tag** | the orange box saying which lesson idea a slide shows. |

Same use as the other glossary slides: leave up, point at it when a word comes up. The words here appear mostly in Part 4.

## Glossary 4 of 4: more words

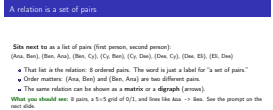
| | |
|---|---|
| **set** | a collection of distinct things, like the five students in the row. |
| **function** | a named piece of code that takes inputs and gives back an answer. |
| **record** | one row of data about one person: an id, a name, maybe an email. |
| **A ~ B** | we write this for "A is similar to B". $\not\sim$ means "is not similar to". |
| **equivalence class** | one group of items that are all the same as each other, such as the records of one person. |
| **comparator** | a function that says which of two items goes first; returns 0 for a tie. |
| **end-to-end test** | a test that runs the whole pipeline, not one piece. |
| **design smell** | a sign in the structure of the code that something is wrong, even if tests pass. |
| **hash** | a short fingerprint computed from data; equal data gives equal fingerprints. |
| **recovery** | the command that puts the project back to a known good step. |
| **drift** | Claude doing something other than what we asked. |

Same use as the other glossary slides. The tilde was introduced here because it appears in the test messages: Jon Smith is similar to John Smith is written with a tilde.

Start with a seating chart, not with notation.

Five students in a row: **Ana**, **Ben**, **Cy**, **Dee**, **Eli**.
Question: *who sits next to whom?*

Put five names on the board in a row before showing anything else. Ask: who sits next to Ben? (Ana and Cy.) Who sits next to Ana? (Ben only.) That is a relation. We have not used any math words yet. Expected: students answer instantly; the point is they already understand this.



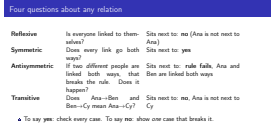

Ask: how many pairs? Predict before the next slide runs it: 8, because 4 neighbor-pairs times 2 directions. Common mistake: counting 4. Say: the order matters, so Ana-Ben and Ben-Ana are separate entries. Pair with prompt 01 on the next slide.

PROMPT FOR CLAUDE prompts/01_seating_relation.md

```
Create demos/seating.py. Students: Ana, Ben, Cy, Dee, Eli (in that order, a list named ITEMS).
Define next_to(a, b) as true when a and b sit directly next to each other, either seats left, right.
Store it as a set of ordered pairs named NEXT_SEAT_TO (as 8 pairs).
Make run as a script, print:
  1. a header naming the task;
  2. a 5x5 matrix of 0/1 with the names as row and column labels;
  3. a digraph as text, one line per student listing all neighbours, like "Ben -> Ana, Cy".
Then run: python3 demos/seating.py   and show me the output.
```

**What you should see:** 8 pairs; a matrix with ones next to the diagonal and zeros on the diagonal; lines Ana -> Ben, Ben -> Ana, Cy . . . Eli -> Dee.
**Recovery:** `cp reference/demos/seating.py demos/` then `python demos/seating.py`
**1 PREDICT (30 s):** How many ones will the matrix have? Which cells on the diagonal?

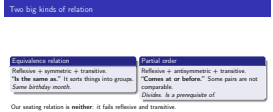
Paste, run, read the output aloud. Predict answer: 8 ones, no ones on the diagonal (nobody sits next to themselves). Explain beat: the diagonal is all zeros because a person is not next to themselves; remember this, it comes back as reflexive. If Claude names the list something else, ask it to rename to ITEMS.




Concrete first, word second. Ask the four plain questions about the seating chart before naming them: Is anyone next to themselves? If Ana is next to Ben, is Ben next to Ana? Can two different people each count as at or before each other? If Ana is next to Ben and Ben next to Cy, is Ana next to Cy? Then attach the names. Symmetric and antisymmetric are not opposites: some relations are both, some neither. Do not dwell; Part 4 tests each one.




Predict answer: symmetric fails; 1 divides 2 but 2 does not divide 1. Point out the printed witness is exactly that pair: a no must come with a concrete example, a yes means no counterexample exists in the items we tried. If Claude's import of ITEMS fails, ask it to rename the student list in seating.py to ITEMS. Important honesty point: checking items is evidence for yes on a small set, not a proof for all numbers.




Ask: in a room, what groups could we sort people into so everyone is in exactly one group? Birthday month. That is an equivalence. Ask: can you think of two things where one has to come before the other? Course prerequisites. That is a partial order. Partial because two courses may have no prerequisite link; neither has to come first.

Sort relations into kinds

PROMPT FOR CLAUDE prompts/10_classify_relations.md

```
Create demos/classify.py using the four checkers in demos/properties.py. Print a table with
one row per relation and one column per property (yes/no), then a one-sentence verdict per relation:
verdict = "equivalence" if reflexive+symmetric+transitive; "partial order" if
reflexive+antisymmetric+transitive; otherwise "neither".
Relations: (1) same birthday (pairs, so 0 if several students given as (name, month) pairs;
(2) "x is at most 3 years older than y", i.e. x - y <= 3; (3) "vote went to" from demos/voting.py;
(4) "x < y" on {1, 2, 3}. Run it and show me the table.
```

**What you should see:** verdicts in order: equivalence, partial order, partial order, neither, neither. Row 5 (<) fails reflexive, so "neither."
**Resource:** `cp reference/demos/classify.py demos/`
**1 PREDICT (30 s):** Is "less than" (<) a partial order? Why or why not?

Predict answer: no, because $3 < 3$ is false, so it is not reflexive. The strict version fails and the at-most version succeeds; this trips up many people. Explain beat: one table, one sentence per row. Do not spend more than six minutes here.

**Equivalence: deduplicating.** Contacts "Jon", "John", "Jonathon Smith". Same person? Group them, keep one. Software does this by rewriting each entry to one standard form (a "hash" or key) and comparing, which only works if "same" is a true equivalence.

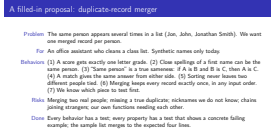
**Partial order: "needs first."** Course prerequisites, build order, which function to test first. If two things need each other there is no valid order: a bug in the plan.

If "same person" is not transitive, a duplicate-record tool gives **different answers depending on the order of the records**.

Real-world first: ask who has had two accounts with the same company, or two entries for one person in their phone. That is the problem we build a fix for. Then prerequisites: ask what happens if course A requires B and B requires A. Nobody can enroll. Those are the two relation kinds in everyday life.




This is the step beginners skip. The proposal is not a formal document; it is half a page. Each behavior sentence will become a test later, so write sentences you could check. Ask the room: what is the idea in one sentence? Elicit: remove duplicate student records.




Read it once, slowly, then ask: which behavior sentences could we check with a yes or no? All of them. Which one do you think will give us trouble? Expected: number 3, the transitivity one, or number 2. It is fine if nobody guesses; we find it with a test. Point at Risks: naming risks before building is the cheap part. Note behavior 1 is a warm-up for bins and boundaries; it is not about names.

## The proposal becomes the test list

| Behavior sentence | Becomes |
|---|---|
| A score gets exactly one letter grade | bins + boundary tests |
| Close first names can be the same person | a match function + tests |
| Same person is a true sameness | transitivity property test |
| A match is the same from either side | symmetry property test |
| Sorting never ties two different people | antisymmetry property test |
| Merge keeps every record once | end-to-end test |
| Know which piece to test first | dependency order |

This slide is the bridge from writing to testing. Each plain sentence on the left becomes a test on the right. If a sentence cannot become a test, it is too vague: rewrite it. Ask: which sentence here is too vague to test? Good discussion: Close first names needs a rule, which we pick in step 06.




Transition: the proposal says what; now we plan how. Planning here means decomposition and order. Keep it short: two minutes.


PROMPT FOR CLAUDE — prompts/30_session_rules.md

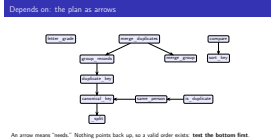
We are building a small Python project live for a class, using test-driven development.
Rules for the whole session:
- Use only the standard library plus pytest. nothing else.
- Work in the current folder. Package target/ holds code, tests/ holds tests, demos/ holds
  stand-alone demo scripts. Create a pytest.ini containing: [pytest] / pythonpath = . /
  testpaths = tests / addopts = -ra -q --tb=short --strict-markers
- Each time I paste a prompt, do exactly that step and nothing more.
- For test-first steps: write the test first, run pytest, show me the output, and stop.
- Do not write the code that makes it pass until I ask.
- Never write code for a later step. Keep every file short and readable.
Reply "ready" and wait.

**What you should see:** Claude replies "ready" (and may create pytest.ini).
**Recovery:** use if Claude jumps ahead. Paste this again.
**1 PREDICT (30 s):** Why do we tell Claude not to write the code yet?

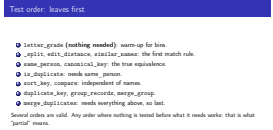
This is the first thing pasted. It makes Claude behave like a careful partner: one step at a time, test first, stop. Predict answer: so we see the test fail first (red) and know the test can fail. If Claude writes ahead later, paste this again; it is the recovery for drift.

| # | Piece | What it promises |
|---|---|---|
| 1 | `letter_grade` | one letter per score; bins and boundaries |
| 2 | `similar_names` | first-draft "looks alike" match |
| 3 | `same_person` | a true equivalence (fix for the draft) |
| 4 | `is_duplicate` | symmetric from both sides |
| 5 | `compare` / `sort_key` | antisymmetric order |
| 6 | `merge_duplicates` | groups, merges, loses nothing |
| 7 | dependency order | which piece to test first |
| 8 | cycle check | refuses an order that cannot exist |

Walk the table top to bottom; one sentence each. Piece numbers match the slide titles in the next section and the prompt files. Point out piece 3 is a fix of piece 2: we will build the bad version on purpose and let a test catch it.




Read the arrows aloud: merge_duplicates needs group_records and merge_group; group_records needs duplicate_key; duplicate_key needs canonical_key; and so on. Letter_grade and compare stand alone. Ask: which function would you test first? Any bottom one: _split, sort_key, letter_grade. That is the partial-order idea: many valid orders, as long as every function is tested after the ones it needs. Prompt 13 later has Claude draw the real graph from our own code.




Emphasize: we are not claiming this is the only order. Ask for a different valid order from the room and check it against the arrows. The workshop rule: if you test a function before the ones it depends on, a red test might blame the wrong function.

Piece 1: letter_grade

| Bin | Scores | Letter |
|---|---|---|
| 1 | 90–100 | A |
| 2 | 80–89 | B |
| 3 | 70–79 | C |
| 4 | 60–69 | D |
| 5 | 0–59 | F |
| 6 | $< 0$ or $> 100$ | error |

**Boundaries to test (both sides):**
59/60, 69/70, 79/80, 89/90, plus 0, 100, −1, 101.

**Why bins?** 101 possible scores, about 14 tests. Same bin, same behavior.

Built live in Part 4, prompt **04**.

Bins are groups of inputs the code treats the same. Pick one test per bin and both sides of each edge, because edges are where programmers make off-by-one mistakes. Ask: if we test 85 and 89, did we learn anything different? No, same bin; but 89 and 90 are different bins. Expected answer for why boundaries: the greater-than versus greater-or-equal mistake lives at the edge.

- same_bin(a, b) means: letter_grade(a) equals letter_grade(b).
- **Reflexive**: every score is in its own bin. **Symmetric**: if a is with b then b is with a. **Transitive**: a with b, b with c, then a with c.
- Test with helper functions that return the **witness** if a property fails, else nothing.
- This test should be **green on first run**: it records a fact.

Built in Part 4, prompt **05**.

Say aloud: This test passes at once. That is allowed here: it records a fact and builds our reusable checkers. Ask: when might a test that never failed be suspicious? When we cannot tell it is able to fail. Here, we know the same checker will catch a bug in the next pieces.

Piece 2: similar_names

| Pair | Expected | Why |
|---|---|---|
| Jon Smith, John Smith | similar | close given names |
| John Smith, Jonathan Smith | similar | close given names |
| Jon Smith, Maria Garcia | not similar | different people |
| Jon Smith, Jon Jones | not similar | family names differ |
| jon smith, JON SMITH | similar | case does not matter |

Rule: same family name, and given-name closeness $\geq 0.5$, where closeness $= 1 -$ edit distance/longer length. **Edit distance** = number of single-letter inserts, deletes or changes to turn one word into the other. Prompt **06**.

This is a reasonable, natural first rule: if the names look alike, call them the same. It will turn out to be a trap. Do not tell the room yet. Ask: does it look right? Expected: yes. Edit distance example: Jon to John is 1 (insert h). Say: we deliberately write the obvious rule so a test can catch it.



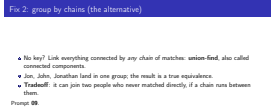

Chain Jon John Jonathan, but Jon and Jonathan are **not** linked. *Transitivity* **fails**, with a concrete witness.
Plan: a property test over the names list that prints the failing triple. Prompt **07**.

This is the key moment. Ask the room to predict before the test runs: will the transitive test pass? Most will say yes. The failure message names the witness triple. Then ask: what would break in the product? Answer: three records for one person, or two, depending on the order we process them. Honest point: nothing was wrong with any single pair; the problem is the relation as a whole.

- **Normalize**: rewrite every name to one standard form (nickname table: jon, john → jonathan).
- Compare the **keys**: equal keys ⇒ same person.
- Equal keys are reflexive, symmetric, transitive **by construction**.
- **Tradeoff**: we only recognize nicknames in our table.
  - The old failing test stays recorded as the witness we fixed.

Prompt **08**.

Why it works: two names are the same exactly when their keys are equal, and equality of keys inherits all three properties from equality itself. Ask: what does it cost? We must know the nicknames. Expected: real systems have a table that grows. Good place to mention that hash-based deduplication works for the same reason.




Union-find in one sentence: start with every item alone; each time two items match, merge their groups. At the end, items in the same group are connected by some chain. Ask: when could that hurt? Chain of similar names that drifts: Jon, John, Joan, Jane. Neither fix is better in general: one is stricter, one is looser. The decision belongs to the person who owns the data.

Piece 4: is_duplicate must be symmetric

| Record A email | Record B email | Bin | Expected |
|---|---|---|---|
| same | same | both present, equal | duplicate |
| x | y | both present, differ | not duplicate |
| x | missing | one missing | **not** duplicate (both directions) |
| missing | missing | both missing | duplicate |

Draft rule: a missing email matches anything from one side only: is_duplicate(a,b) ≠ is_duplicate(b,a). The symmetry test prints the pair. Prompt **10**.

Concrete: record A has an email, record B does not. The lazy rule says A matches B but B does not match A. Ask: what happens if I sort the list differently? Sometimes we find the duplicate, sometimes not. Four bins for two fields, plus a reflexive test. Point at the symmetric test printing both records.

| Bin | Example | Expected |
|---|---|---|
| different family names | Garcia vs Smith | Garcia first |
| same family, different given | Jon Smith vs John Smith | by given name |
| same name, different id | two "Jon Smith"s | by id (no ties) |

A comparator returning 0 for *different* people lets the result depend on input order: antisymmetry fails. Fix: add a unique tie-breaker (family, given, id). Prompt **11**.

Plain version: if A is at or before B and B is at or before A, they had better be the same record. Ask: what goes wrong in a sorted list when two different people compare equal? Their order depends on how the list arrived. Expected: students may say it is a harmless tie; answer: it makes output unrepeatable, which makes tests flaky.

Piece 6: `merge_duplicates`

| Bin | Input | Expected |
|---|---|---|
| empty list (boundary) | `[]` | `[]` |
| no duplicates | distinct people | one merged record each |
| one group | Jon, John, Jonathan (same email) | one record, ids merged |
| mixed | some duplicates, some not | only true duplicates merge |
| shuffled input | same data, any order | same output |

The groups of records are the equivalence classes. Prompt **12**.

This is the payoff: all earlier properties make this test boring, which is the goal. Ask: why does the shuffled-input test matter? It is the symptom of a missing property: it fails exactly when transitivity or symmetry is broken.

- A tool reads our own source, finds which function calls which, and returns an order with **leaves first**.
- Test plan: the graph for a small sample module matches a written expected list; the order never puts a function before something it needs.

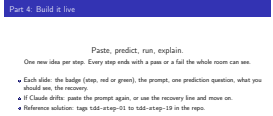
Prompt **13**.

Two minutes. The idea: the arrows we drew by hand can be computed. The dependency order is a partial order: if A needs B and B needs C then A needs C (transitive), and as long as no two functions need each other (no cycle) it is antisymmetric.




If parse needs validate and validate needs parse, there is **no leaf to test first**. The order is not antisymmetric. The code should say so: `dependency cycle: parse -> validate -> parse`. Prompt **14**.

Real life: course A requires B, and B requires A; or two modules each importing the other. Ask: where would you start testing? Nowhere. That is why this is a smell. The fix is a design change, not a better test.




Settle the room. Remind them: red is a good sign. Say you will not hurry the predict beat. If time gets tight, drop the alternative fix (prompt 09) and the cycle step (prompt 14); the core line is 04 to 08, 10 to 12. Ladder tags are the safety net, never a first choice; the point is watching code grow.




Predict answer: every test fails because the function only raises NotImplementedError. Explain beat: this is a red we asked for. The stub lets the tests run so the failure is about our function, not about a missing file. Show one failing line and the E line at the bottom. If Claude also writes the implementation, ask it to revert it and stop.




Predict answer: yes if the implementation is the simplest correct one; the printed bins are the five we planned. Explain beat: 101 possible scores, a handful of tests, and we trust the whole range because same bin behaves the same. Ask: why did we test 59 and 60 and not 55 and 56? Boundary.




Predict answer: green. Many students expect red. Explain that tests can record a fact that is already true; what matters is that the checker functions we just wrote (reflexive_failure and friends) will fail loudly in later steps. Ask: how could we check these helpers can fail? In the next steps they will.




Predict answer: red, NotImplementedError, as in step 1. Second time through the loop should feel faster. Ask a student to predict aloud without looking at notes. Explain beat: the tests describe what similar means: Jon and John yes, Jon and Maria no, Jon Smith and Jon Jones no.




Predict answer: many will say True. It prints False: edit distance between Jon and Jonathan is 5 over a longer length of 8, closeness 0.375, under 0.5. Explain beat: every pair test is green, yet the whole relation has a hole. That tension is the lesson; do not resolve it yet.




This is the key red. Predict answer: the transitive test fails; reflexive and symmetric pass. Explain beat: a witness is the concrete triple that proves a no. Ask: if we sorted records in a different order, what would the merger do? Different groupings. Keep this failing test; it is our recorded witness.



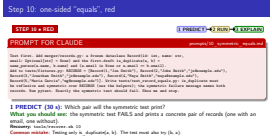

Predict answer: yes, because equal keys are an equivalence by construction. Explain beat: the same test that was red is green, with the same Jon, John, Jonathan. Tradeoff to say aloud: we only know the nicknames in our table. Ask: what happens with Jonny? Not in the table, so it stays separate.




Optional step; skip if time is short. Predict answer: red, same pattern. Explain beat: this is the second way to get an equivalence: link by chains, then group. Ask: which fix is stricter? The key. Which is looser? Chains.




Predict answer: three groups. Explain beat: Jon and Jonathan end up together even though they never matched directly, because John links them. Ask: when is that wrong? A chain of drifting names, Jon, John, Joan, Jane. The tradeoff is real; the owner of the data decides.




Predict answer: a record with an email against a record without one. The lazy rule treats a missing email as a wildcard from one side only. Explain beat: the witness is two records, shown in both directions, giving different answers. Ask: why is that a problem for a merge? The result depends on which record is first.




Predict answer: False both ways. Explain beat: swapping a and b cannot change the answer. Ask: what other relation in daily life is symmetric by design? Is the same age as, sits next to.


Step 12: sort ties, red

STEP 12 ● RED

1 PREDICT → 2 RUN → 3 EXPLAIN

PROMPT FOR CLAUDE

prompts/12_antisymmetric_sort.md

Your task: Add antisymmetry to sort with the first-draft comparator, %1 for Records: compare the family name only (last word of name, lowercase), returning -1, 0 or 1.

Write tests/test_ordering.py using BEGINNER and the helpers, with at_or_before(a, b) meaning comparator(a, b) <= 0: (1) at_or_before is antisymmetric (message names the two records); (2) sort_records(people) is idempotent (sorting twice equals sorting once); (3) exactly one SAME line for every permutation of BEGINNER (itertools.permutations). Run pytest. Expect the antisymmetric and the shuffle tests to fail. Show me and stop.

**1 PREDICT (30 s):** Which two names will the witness show?

**What you should see:** the antisymmetric test FAILS; the witness is Jon Smith and John Smith (same family name).

**Recovery:** tools/reverser.sh 12

**Common mistake:** Thinking a tie is harmless. Two different people comparing equal makes the sort depend on input order.

Predict answer: two Smiths, because the comparator looks only at the family name. Explain beat: the comparator returns 0 for different people. Antisymmetric means if a is at or before b and b at or before a then they are the same. Here two different people are each at or before the other.




Predict answer: Garcia, then Jon, John, Jonathan Smith (by given name). Explain beat: adding id as a final tie-breaker makes the key unique, so no two different people tie. Ask: what would happen with two identical names? Id breaks the tie.




Predict answer: red. Explain beat: all pieces are individually green; this test checks they work together. Ask: where would a failure here point us? The new code only, because the pieces under it were already tested.




Predict answer: four merged records. Explain beat: Jon had no email so the symmetric rule keeps him separate. Ask: is that right? It follows the rule we chose; the proposal said risks include missing a duplicate. A tradeoff we can name.




Predict answer: red, same pattern. Explain beat: this tool draws the same arrows we drew on the plan slide, from the real code. Ask the room whether the computed graph will match their picture.



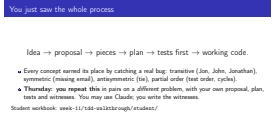

Predict answer: a leaf first, merge_duplicates last. Explain beat: this is leaves-first. Compare with the dependency drawing in Part 3; they agree.



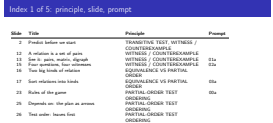

Predict answer: red. Explain beat: we ask the tool to refuse an order that cannot exist and to name the loop. Ask: why must it name the loop? A bare error is not actionable.




Predict answer: the loop spelled out. Explain beat: the witness is the path. All tests are green: stop and look at the git log, 19 small steps. Ask: which step surprised you most?




Close the loop. Ask: which relation property do you now think you could find a bug with? Take three answers. Then the bridge: on Thursday you do exactly this process yourselves on a different but parallel problem, in pairs. Say this reassuringly: the same five steps, the same loop, the same legend. The Thursday run-of-show and facilitator page are in the instructor folder.



Index 1 of 5: principle, slide, prompt

| Slide | Title | Principle | Prompt |
|---|---|---|---|
| 2 | Predict before we start | TRANSITIVE TEST, WITNESS / COUNTEREXAMPLE | |
| 12 | A relation is a set of pairs | WITNESS / COUNTEREXAMPLE | |
| 13 | See it: pairs, matrix, digraph | WITNESS / COUNTEREXAMPLE | 01a |
| 15 | Four questions, four witnesses | WITNESS / COUNTEREXAMPLE | 01b |
| 16 | Two big kinds of relation | EQUIVALENCE VS PARTIAL ORDER | |
| 17 | Sort relations into kinds | EQUIVALENCE VS PARTIAL ORDER | 02a |
| 23 | Rules of the game | PARTIAL-ORDER TEST ONTOLOGY | 02c |
| 25 | Depends on: the plan as arrows | PARTIAL-ORDER TEST ORDERING | |
| 26 | Test order: leaves first | PARTIAL-ORDER TEST ORDERING | |

Use this to find a slide fast: slide number, which lesson idea it shows, and which prompt file it pairs with (prompts/NN, a = first block, b = second block).

| Slide | Title | Principle | Prompt |
|---|---|---|---|
| 27 | Piece 1: letter_grade | EQUIVALENCE PARTITIONING, BOUNDARIES | |
| 28 | Piece 1b: "same list" is an equivalence | REFLEXIVE / SYMMETRIC / TRANSITIVE TESTS | |
| 29 | Piece 2: similar_names | EQUIVALENCE PARTITIONING | |
| 30 | Piece 3: same_person must be transitive | TRANSITIVE TEST, WITNESS / COUNTEREXAMPLE | |
| 31 | Fix 1: a canonical key | EQUIVALENCE PARTITIONING, NOTHING / COUNTEREXAMPLE | |
| 32 | Fix 2: group by chains (the alternative) | EQUIVALENCE PARTITIONING, WITNESS / COUNTEREXAMPLE | |
| 33 | Piece 4: is_duplicate must be symmetric | REFLEXIVE AND SYMMETRIC TESTS | |
| 34 | Piece 5: ordering must be antisymmetric | ANTISYMMETRIC TEST | |
| 35 | Piece 6: merge_duplicates | WITNESS / COUNTEREXAMPLE, EQUIVALENCE PARTITIONING | |

Use this to find a slide fast: slide number, which lesson idea it shows, and which prompt file it pairs with (prompts/NN, a = first block, b = second block).

| Slide | Title | Principle | Prompt |
|---|---|---|---|
| 36 | Piece 7: depends-on order | PARTIAL ORDER TEST, ORDERING | |
| 37 | Piece 8: cycles are a design smell | ANTISYMMETRIC TEST, WITNESS / COUNTEREXAMPLE | |
| 39 | Step 1: bins and boundaries, red | EQUIVALENCE PARTITIONING, BOUNDARIES | 04a |
| 40 | Step 2: bins and boundaries, green | EQUIVALENCE PARTITIONING, BOUNDARIES | 04b |
| 41 | Step 3: the bins are an equivalence | REFLEXIVE / SYMMETRIC / TRANSITIVE TEST | 05a |
| 42 | Step 4: the fuzzy name rule, red | EQUIVALENCE PARTITIONING | 06a |
| 43 | Step 5: the fuzzy name rule, green | EQUIVALENCE PARTITIONING | 06b |
| 44 | Step 6: the transitivity test, red | TRANSITIVE TEST, WITNESS / COUNTEREXAMPLE | 07a |
| 45 | Step 7: fix with a canonical key, green | EQUIVALENCE PARTITIONING, WITNESS / COUNTEREXAMPLE | 08a |

Use this to find a slide fast: slide number, which lesson idea it shows, and which prompt file it pairs with (prompts/NN, a = first block, b = second block).

| Slide | Title | Principle | Prompt |
|---|---|---|---|
| 46 | Step 9: chains (union-find), red | EQUIVALENCE PARTITIONING, WITNESS / COUNTEREXAMPLE | 09a |
| 47 | Step 9: chains (union-find), green | EQUIVALENCE PARTITIONING, WITNESS / COUNTEREXAMPLE | 09b |
| 48 | Step 10: sort-tabled "equals", red | REFLEXIVE AND SYMMETRIC TESTS, WITNESS | 10a |
| 49 | Step 11: symmetric equals, green | REFLEXIVE AND SYMMETRIC TESTS, WITNESS | 10b |
| 50 | Step 12: sort ties, red | ANTISYMMETRIC TEST | 11a |
| 51 | Step 13: unique sort key, green | EQUIVALENCE PARTITIONING, ANTISYMMETRIC TEST | 11b |
| 52 | Step 14: merge end to end, red | EQUIVALENCE PARTITIONING, WITNESS / COUNTEREXAMPLE | 12a |
| 53 | Step 15: merge end to end, green | EQUIVALENCE PARTITIONING | 12b |
| 54 | Step 16: depends-on order, red | ORDER-ORDER TEST ORDERING | 13a |

Use this to find a slide fast: slide number, which lesson idea it shows, and which prompt file it pairs with (prompts/NN, a = first block, b = second block).

| Slide | Title | Principle | Prompt |
|---|---|---|---|
| 55 | Step 17: depends-on order, green | PARTIAL-ORDER TEST ORDERING | 11b |
| 56 | Step 18: cycle detection, red | ANTISYMMETRIC TEST, WITNESS / COUNTEREXAMPLE | 16a |
| 57 | Step 19: cycle detection, green | ANTISYMMETRIC TEST, WITNESS / COUNTEREXAMPLE | 16b |

Use this to find a slide fast: slide number, which lesson idea it shows, and which prompt file it pairs with (prompts/NN, a = first block, b = second block).